

Laborator 4 Arhitecturi Hardware Reconfigurabile

Utilizarea primitivelor în VHDL. Memoria BlockRAM

Conținut laborator:

1. **Obiectivul lucrării.**
2. **Noțiuni teoretice:**
3. **Cerințe laborator.**
4. **Temă.**

1. Obiectivul lucrării:

În acest laborator se vor utiliza și alte resurse ale circuitelor reconfigurabile FPGA alături de blocurile logice configurabile. Memoriile BRAM sunt memorii interne din FPGA care lucrează la viteze foarte mari și permit stocarea unui volum mai mare de date. Ele au o organizare care poate fi configurată, putând fi, opțional, memorii uniport sau memorii biport. Alocarea acestor memorii nu are loc implicit ci numai prin utilizarea primitivelor, în care se specifică în mod explicit configurația memoriei. Primitivele sunt componente declarate de fabricanții circuitului și care sunt utilizate ca atare.

În acest laborator se va implementa un sistem de afișare VGA care utilizează memoria BRAM pentru stocarea imaginii care se dorește afișată.

Utilitatea laboratorului: Resursele accesibile prin primitive printre care se numără memoriile BRAM și administratorul de ceas, sunt utile în numeroase aplicații. Acest laborator este dedicat în primul rând modului în care se pot utiliza resursele suplimentare, prin declararea primitivelor, pe care le posedă un circuit FPGA. Acestea în cazul în care nu sunt alocate rămân neutilizate, ele nu se vor alocat printr-o comandă implicită. Deci un obiectiv suplimentar pe care și-l propune acest laborator este acela de a indica o metodă optimizată de abordare a circuitelor reconfigurabile în sensul utilizării corecte a resurselor de care dispun.

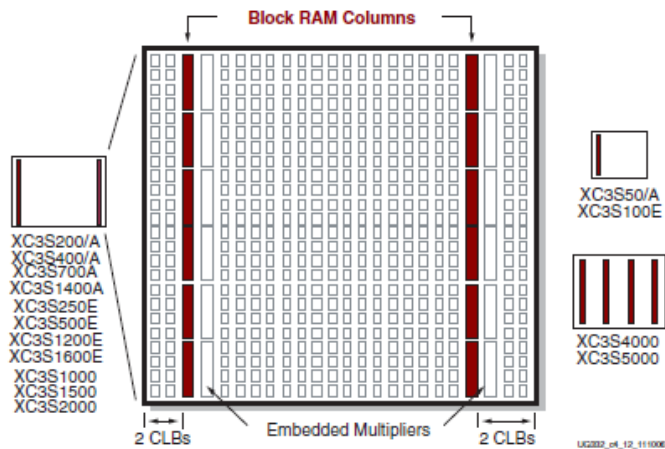
2. Noțiuni teoretice

Memoriile BlockRAM

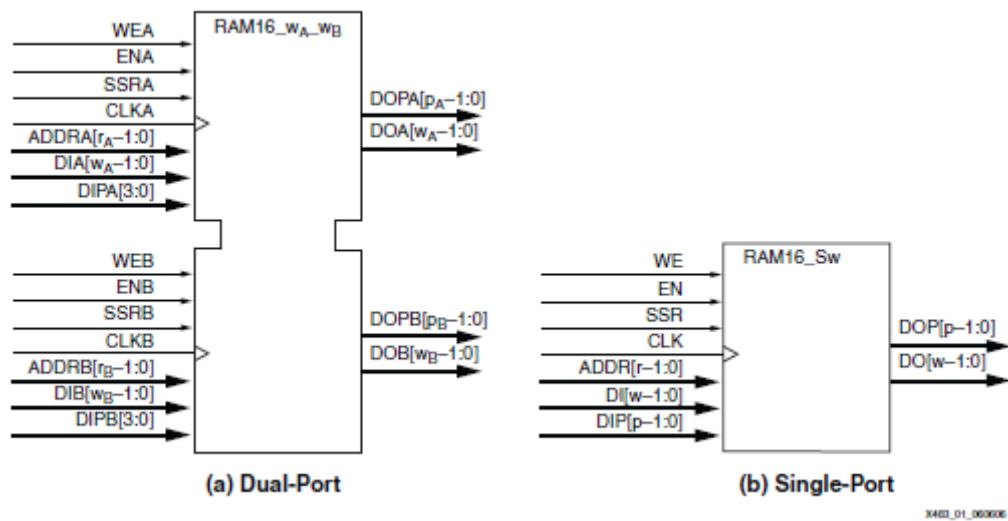
Alături de memoria RAM distribuită care se poate obține din blocurile logice configurabile utilizând perechile de slice-uri M circuitele FPGA posedă și memorie RAM dedicată, internă, care se poate utiliza la aplicații care solicită spații de memorie onchip mari.

Memoria Block Ram (BRAM) este o memorie de tipul SRAM care poate fi configurată să devină memorie uni port sau memorie biport, memorie de tipul ROM, memorie de tipul RAM sau FIFO, tabele de funcții extinse, bufferi circulari sau regiștrii de deplasare. De asemenea se pot configura numărul de locații și lățimea unei locații (numărul de biți). În cazul familiei de FPGA-uri Spartan 3, există blocuri de memorie de câte 18k din care 16k sunt alocați pentru stocarea datelor iar 2k pentru stocarea biților de paritate. Biții de paritate nu sunt generați în vreun fel de logica internă existentă în FPGA, ei pot fi utilizați tot ca biți de date aflați însă în locații separate și acționabili pe magistrale separate. La circuitul Spartan3 XC3S400 de exemplu există 16 memorii BRAM deci are o capacitate de stocare internă de 288kbiți.

Fiecare memorie BRAM este una sincronă configurabilă biport sau cu un singur port, având structura prezentată mai jos. Acestea sunt amplasate fizic pe laturile circuitului FPGA așa cum se poate vedea în figură:



Fluxul de date diferă, în funcție de modul în care memoria este configurată.



Memoriile biport există în structura internă a FPGA-ului ca entități distincte. Acestea pot fi apelate utilizând modul de lucru cu „primitive” așa cum se procedează și cu administratorul de ceas.

Modul în care o memorie biport este identificată este prezentat în figura de mai jos:

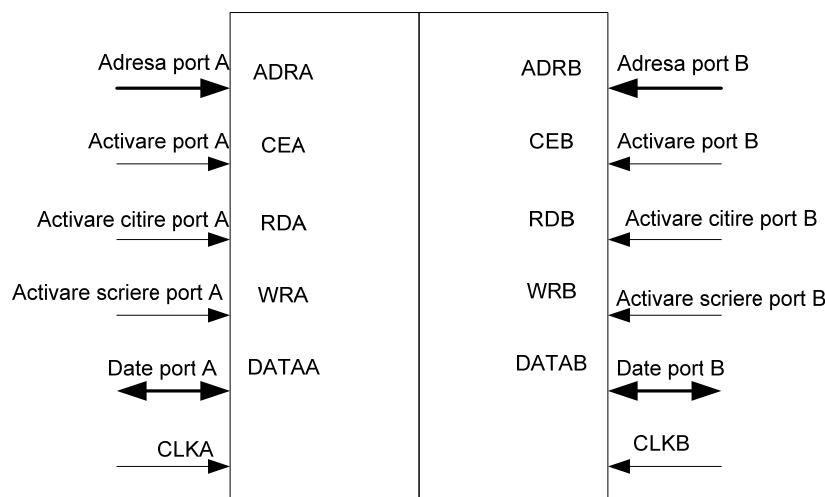


Fig.4.5. Memoria biport

Memoria biport este o structură care permite operarea simultană pe două direcții (scris – citit sau citit – scris) a locațiilor care le deține. Fiecare locație poate fi adresată, citită sau scrisă în același timp cu adresarea, citirea sau scrierea unei alte locații. Orice operație se desfășoară sincron, deci în ritmul unui semnal de ceas.

Memoria posedă două magistrale de adrese, pentru portul A și portul B, două comenzi de activare, două comenzi de validare scriere și două comenzi de validare citire. De asemenea, fiecare port în parte va avea propria magistrală de date, de intrare sau ieșire.

Circuitul Spartan 3 3s400fg456 posedă 16 blocuri de memorie biport având fiecare a câte 16kb. Fiecare port în parte poate fi configurat pentru o dimensiune a magistralei de adrese și a magistralei de date dorită, conform tabelului de mai jos:

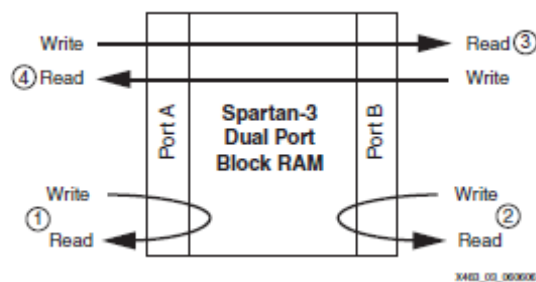
Tabela 4.1. Configurații posibile la memoria biport

Număr locații	Dimensiune locație (biți)
16k	1
8k	2
4k	4
2k	8
2k	8+1(paritate)

1k	16
1k	16+2(paritate)
512	32
512	32+4(paritate)
256	72(numai un singur port)

Cu excepția ultimei linii toate celelalte combinații pot fi utilizate pentru configurarea ambelor porturi în orice formă. Această diferențiere în configurarea porturilor este foarte avantajoasă în cazul de față. Spre exemplu, datele preluate de la cardul SD sunt transmise către memoria internă program sau obiecte imagine bit cu bit, datorită comunicației seriale. Pe de altă parte, din memoria de program procesorul grafic își preia datele instrucțiune cu instrucțiune, adică 16 biți într-un singur ceas.

Deși este posibilă orice combinație de adrese/date și orice operație pe un port la un moment dat, totuși la un moment dat se poate opera o secvență de scriere – citire sau citire – scriere simultan. De exemplu, dacă se lucrează cu un port în modul scriere atunci cu celălalt port se poate lucra în modul citire. Este validă și combinația inversă. Însă dacă, la un moment dat, se dorește utilizarea aceluiași port pentru scriere și pentru citire trebuie să se opteze pentru o configurație uniport.



Alocarea unei memorii BRAM în limbajul VHDL se face prin declararea unei primitive în prezența librăriei UNISIM.

```
library UNISIM;
use UNISIM.VComponents.all;
```

```
component RAMB16_S1_S1
```

```

-- pragma translate_off
generic (
-- "Read during Write" attribute for functional simulation
    WRITE_MODE_A : string := "READ_FIRST" ; -- WRITE_FIRST(default)/
    READ_FIRST/ NO_CHANGE
    WRITE_MODE_B : string := "READ_FIRST" -- WRITE_FIRST(default)/
    READ_FIRST/ NO_CHANGE
-- RAM initialization ("0" by default) for functional simulation: see example
);
-- pragma translate_on
port (
    DIA : in std_logic_vector (0 downto 0);
    ADDRA : in std_logic_vector (13 downto 0);
    ENA : in std_logic;
    WEA : in std_logic;
    SSRA : in std_logic;
    CLKA : in std_logic;
    DOA : out std_logic_vector (0 downto 0);
--
    DIB : in std_logic_vector (0 downto 0);
    ADDR_B : in std_logic_vector (13 downto 0);
    ENB : in std_logic;
    WEB : in std_logic;
    SSRB : in std_logic;
    CLKB : in std_logic;
    DOB : out std_logic_vector (0 downto 0)
);
end component;

```

Alături de porturi, în lucrul cu primitiva BRAM există mai multe atribute de declarat.

- **Numărul de porturi și organizarea.** Așa cum s-a arătat fiecare bloc de memorie BRAM poate fi configurat să lucreze uniport sau biport. Specificarea numărului

de porturi se face în primitivă prin numele componentei. La Spartan 3 pentru memoria uniport declarația componentei este prezentată în tabelul de mai jos:

Organization	Memory Depth	Data Width	Parity Width	DI/DO	DIP/DOP	ADDR	Single-Port Primitive	Total RAM Kbits
512x36	512	32	4	(31:0)	(3:0)	(8:0)	RAMB16_S36	18K
1Kx18	1024	16	2	(15:0)	(1:0)	(9:0)	RAMB16_S18	18K
2Kx9	2048	8	1	(7:0)	(0:0)	(10:0)	RAMB16_S9	18K
4Kx4	4096	4	-	(3:0)	-	(11:0)	RAMB16_S4	16K
8Kx2	8192	2	-	(1:0)	-	(12:0)	RAMB16_S2	16K
16Kx1	16384	1	-	(0:0)	-	(13:0)	RAMB16_S1	16K

Așa cum se poate observa, numele primitivei variază cu organizarea memoriei uniport.

La memoriile biport declarațiile variază de asemenea cu organizarea memoriilor astfel:

		Port A					
		16Kx1	8Kx2	4Kx4	2Kx9	1Kx18	512x36
Port B	16Kx1	<u>_S1_S1</u>					
	8Kx2	<u>_S1_S2</u>	<u>_S2_S2</u>				
	4Kx4	<u>_S1_S4</u>	<u>_S2_S4</u>	<u>_S4_S4</u>			
	2Kx9	<u>_S1_S9</u>	<u>_S2_S9</u>	<u>_S4_S9</u>	<u>_S9_S9</u>		
	1Kx18	<u>_S1_S18</u>	<u>_S2_S18</u>	<u>_S4_S18</u>	<u>_S9_S18</u>	<u>_S18_S18</u>	
	512x36	<u>_S1_S36</u>	<u>_S2_S36</u>	<u>_S4_S36</u>	<u>_S9_S36</u>	<u>_S18_S36</u>	<u>_S36_S36</u>

Deci, prin numele primitivei se specifică atât organizarea cât și porturile pe care aceasta le are. De exemplu, declarația

component RAMB16_S1_S1

desemnează o memorie biport, în care portul A și portul B au organizarea 16kx1.

În cazul în care sunt utilizate două porturi cu organizare diferită la o memorie biport, formulele de calcul care stabilesc o corespondență între adresele unui port și a celui alt port sunt de felul următor:

$$START_ADDRESS_X = INTEGER\left(\frac{ADDRESS_Y \cdot WIDTH_Y}{WIDTH_X}\right)$$

$$END_ADDRESS_X = INTEGER\left(\frac{((ADDRESS_Y + 1) \cdot WIDTH_Y) - 1}{WIDTH_X}\right)$$

- **Inițializarea conținutului.** Implicit, memoriile sunt inițializate cu 0. Totuși, numeroase aplicații practice necesită utilizarea unei inițializări cu alte valori a conținutului memoriei.

Pentru aceasta se utilizează atributul INIT_xx și INITP_xx pentru inițializarea conținutului de date sau de paritate.

Attribute	From	To
INIT_00	255	0
INIT_01	511	256
INIT_02	767	512

Există în total 64 de attribute INIT_xx care se pot inițializa, fiecare desemnând un sector de 256 de biți din memorie. De exemplu: INIT_00 desemnează sectorul care corespunde biților 0 până la 255, INIT_01 desemnează sectorul care conține bitul 256 ... 511 ș.a.m.d. Ultimul sector, accesat prin atributul INIT_3F, desemnează ultimii 256 de biți din spațiul de date.

Spre exemplu, dacă se dorește ca primul octet din memorie să fie 00h în timp ce toți ceilalți octeți din primul sector să fie ffh atunci se scrie:

```
attribute INIT_00 : string;
attribute INIT_01 : string;
```

```
attribute INIT_00 of bram0 : label is
    "ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff00";    --256
attribute INIT_01 of bram0 : label is
    "ffffffffffffffffffffffffffffffffffffffffffffffffffffffffffff";    --512
```

Pentru zona de biți de paritate există atributul INITP_xx care poate avea 8 valori: INITP_00 până la INITP_07 fiecare reprezentând de asemenea un sector de 256 de biți aflați în zona de paritate.

Cerințe laborator:

În acest laborator se va construi, utilizând limbajul VHDL , un circuite care comandă portul VGA de pe machetă. Acesta va prelua o imagine, descrisă sub forma unui șir de pixeli, aflată în memoria BRAM și o va transmite la un monitor prin interfața VGA.

Schema sistemului este prezentată în figura de mai jos.

Schema este compusă din două module, corelate între ele. Unul este responsabil cu generarea semnalelor de sincronizare pe orizontală și pe verticală. Cele două semnale care acest modul le va genera în final vor fi Vsinc și Hsinc.

Cel de-al doilea modul este cel responsabil cu afișarea informațiilor propriu zise (de culoare).

Pentru semnalele de sincronizare se ține cont de faptul că frecvența unui cadru trebuie să fie mai mare de 50Hz. În funcție de numărul de linii ales se obține și frecvența de comutare pe linii.

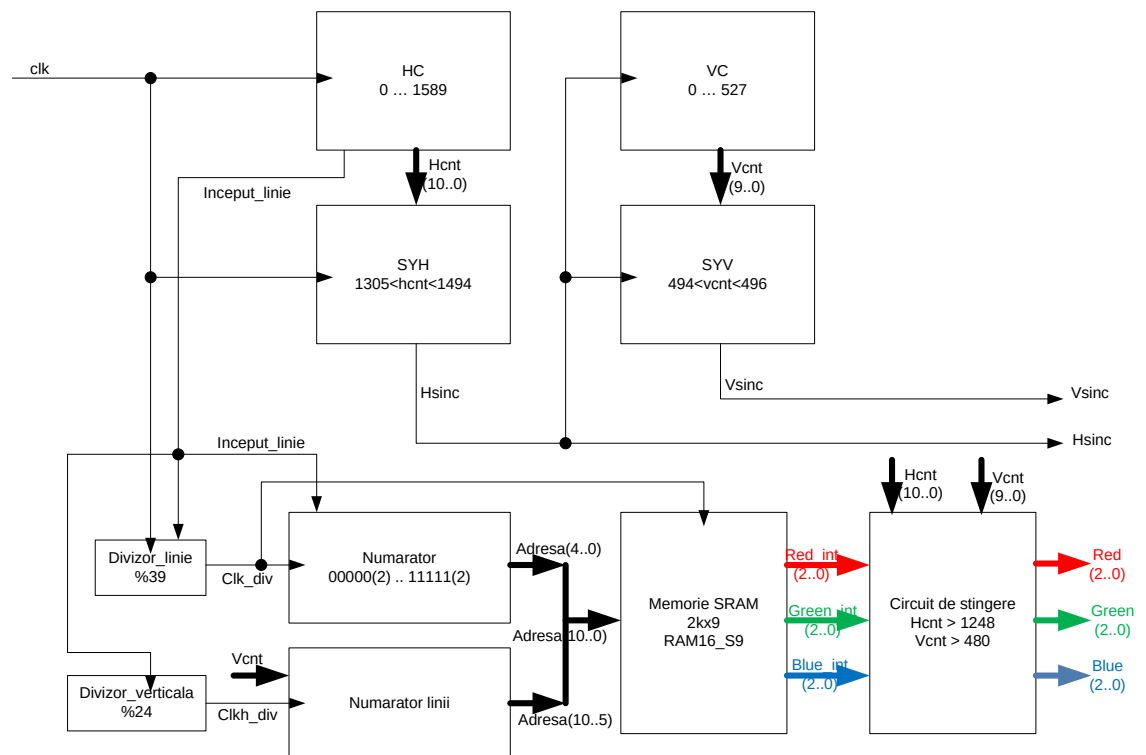


Fig.1 Sistem de afișare utilizând portul VGA cu memorie BRAM

Modulul de afișare a informațiilor pe pixel se bazează pe o memorie BRAM uniport cu organizarea 2kx9. Justificarea alegerii acestei organizări este datorită faptului că, în final, trebuie generate 3 semnale de roșu, verde și albastru pe 3 biți fiecare.

Memoria BRAM este configurată, în acest caz, în forma 8 biți de date și 1 bit de paritate pe fiecare locație. Distribuția informației de culoare este în felul următor:



Fig.2. Distribuția informației de culoare

Deci scrierea unei locații în memoria SRAM și a bitului de paritate care corespunde locației are drept efect programarea unui pixel.

Fiecare linie a fost împărțită în acest exemplu în 32 de părți care pot fi „colorate” în mod diferit în timp ce fiecare cadru (image completă) conține 20 de linii. Pentru aceasta au fost utilizate cele două numărătoare și divizoarele de frecvență care asigură semnalul de ceas ale acestora. La ieșirea celor două numărătoare va rezulta o adresă fizică care va constitui adresa pixelului care se dorește afișat.

Evident, ulterior se poate cu ușurință extinde rezoluția pe orizontală sau pe verticală.

Etape în construirea sistemului:

1. Se va crea un proiect în ActiveHDL, cu circuitul Spartan 3 asociat. (vezi anexa).

2. Se va crea un fișier VHDL (video1.vhd) în care se vor descrie toate modulele prezentate în schema din figura 1 după cum urmează.
Sistemul va avea structura la nivel de porturi prezentată în figura 3.



Fig.3. Pinii sistemului de afișare prin VGA realizat pentru macheta de laborator

Pin	Descriere
Clk	Intrare. Semnal de ceas care realizează sincronizarea modulelor din sistem
Reset	Intrare. Semnal prin care se realizează inițializarea modulelor componente ale sistemului de afișare
Red(2..0), Green(2..0), Blue(2..0)	Ieșiri. Semnale digitale de culoare care apoi sunt convertite pe machetă pentru a deveni semnale analogice prin care este comandat dispozitivul de afișare (monitorul) prin interfața VGA
Hsinc	Buffer. Semnal care realizează sincronizarea pe orizontală a imaginii. Deoarece acest semnal este utilizat și în alte blocuri se declară de tip buffer.
Vsinc	Semnal care realizează sincronizarea pe verticală a imaginii

- a. Declararea primitivei pentru alocarea memoriei BRAM cu organizarea 2kx9

Se face imediat după denumirea arhitecturii, în secțiunea declarării componentelor, înainte de „begin”:

```
--DECLARATIE COMPONENTE PRIMITIVE (MEMORIE BRAM CU ORGANIZAREA 1K X 8+1)
component RAMB16_S9
```

```
-- pragma translate_off
generic (
-- "Read during Write" attribute for functional simulation
    WRITE_MODE : string := "READ_FIRST" -- WRITE_FIRST(default)/ READ_FIRST/
NO_CHANGE
-- RAM initialization ("0" by default) for functional simulation: see example
);
-- pragma translate_on
port (
    DI   : in std_logic_vector (7 downto 0);
    DIP  : in std_logic_vector (0 downto 0);
    ADDR : in std_logic_vector (10 downto 0);
    EN   : in std_logic;
    WE   : in std_logic;
    SSR  : in std_logic;
    CLK  : in std_logic;
    DO   : out std_logic_vector (7 downto 0);
    DOP  : out std_logic_vector (0 downto 0)
);
end component;
--DECLARATIE ADTRIBUTE CARE SE VOR UTILIZA APOI
attribute INIT_00 : string;
attribute INIT_01 : string;
attribute INIT_02 : string;
attribute INIT_03 : string;
attribute INIT_04 : string;
attribute INIT_05 : string;
attribute INIT_06 : string;
attribute INIT_07 : string;
attribute INIT_08 : string;
attribute INIT_09 : string;
attribute INIT_0a : string;
attribute INIT_0b : string;
attribute INIT_0c : string;
attribute INIT_0d : string;
attribute INIT_0e : string;
attribute INIT_0f : string;
attribute INIT_10 : string;
attribute INIT_11 : string;
attribute INIT_12 : string;
attribute INIT_13 : string;
attribute INITP_00 : string;
attribute INITP_01 : string;
attribute INITP_02 : string;
attribute INITP_03 : string;
attribute INITP_04 : string;
attribute INITP_05 : string;
attribute INITP_06 : string;
attribute INITP_07 : string;

--INITIALIZARE CONTINUT MEMORIE BRAM
attribute INIT_00 of bram_img : label is
"38383838C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C738383838";
attribute INIT_01 of bram_img : label is
"38383838C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C7C738383838";
attribute INIT_02 of bram_img : label is
```


Așa cum s-a prezentat în breviarul teoretic, alături de pinii de port, există și alte elemente importante în componenta BRAM. În exemplu de față o componentă importantă o constituie atributele INIT_xx și INITP_xx care indică valoarea inițială pentru date și biți de paritate care există în memorie. Prin aceste atribute se înscrie în memorie imaginea care se dorește afișată în cazul nostru. Formatul acesteia respectă distribuția culorii din figura 2. În zona memoriei alocată datelor se află informația de roșu, de verde și doi biți din informația de albastru. Cel de-al treilea bit de albastru se află în zona biților de paritate (inițializată cu INITP_xx).

În cazul nostru s-a dorit afișarea pe un fundal constituit dintr-un chenar de culoare verde și un fond mov (rosu cu albastru) a unui mesaj scris. Pentru un efect de transparenta la scris s-a utilizat un fond constituit din rosu și albastru nesaturat (valoarea 011). Scrisul propriu zis s-a realizat prin setarea bitului cel mai semnificativ al informației de albastru care a dus la saturarea pe fond și la adăugarea unei nuanțe de albastru pe chenar. Astfel, pixelii care s-au dorit scriși au fost setați în 1 logic în zona de paritate a memoriei, acolo unde se află bitul cel mai semnificativ de albastru.

b. Declararea semnalelor care se utilizează în sistem

În conformitate cu figura 1, se declară următoarele semnale (înainte de begin la arhitectură)

--DECLARATIE SEMNALE

signal hcnt : std_logic_vector(10 downto 0);

signal vcnt : std_logic_vector(9 downto 0);

signal red_int : std_logic_vector(2 downto 0);

signal green_int : std_logic_vector(2 downto 0);

signal blue_int : std_logic_vector(2 downto 0);

signal adresa : std_logic_vector(10 downto 0);

signal clk_div : std_logic;

signal clkh_div : std_logic;

signal data_out : std_logic_vector(7 downto 0);

signal datap_out : std_logic_vector(0 downto 0);

signal inceput_linie : std_logic;

Acestea se utilizează pentru interconectarea modulelor din sistem.

Următoarele module se introduc în interiorul arhitecturii.

c. Descrierea numărătorului de sincronizare pe orizontală

Modulul reprezentat cu HC (Horizontal Counter) în figura 1 este de fapt un numărător care, în ritmul semnalului de ceas de 50MHz numără pentru obținerea a 1589 de puncte pe orizontală. Precizăm că circuitul este conceput să funcționeze atât pe monitoare LCD noi dar și pe mai vechile monitoare cu CRT. Din acest motiv anumite elemente legate de afișare pe CRT au fost păstrate.

Trebuie precizat că nu toate punctele sunt informație afișabilă. O parte din acestea sunt utilizate pentru cursa inversă pe orizontală. Pe durata cursei inverse pe orizontală nu trebuie afișată informația pe ecran.

```
hc:process (clk,reset)
begin
    if reset='0' then
        hcnt <= "000000000000";
        inceput_linie <= '0';
    elsif clk'event and clk = '1' then
        if hcnt < 1589 then
            hcnt <= hcnt + 1;
            inceput_linie <= '0';
        else
            hcnt <= "000000000000";
            inceput_linie <= '1';
        end if;
    end if;
end process;
```

Din punct de vedere al descrierii în VHDL, numărătorul de pixeli pe orizontală este descris ca un numărător până la 1589 cu reluarea numărării. Semnalul `inceput_linie` joacă rol de indicator reîncărcare. Reset-ul este unul asincron.

d. Descrierea numărătorul de sincronizare pe verticală

Numărătorul de sincronizare pe verticală (VC - Vertical Counter) are o structură similară cu numărătorul de pixeli.

```
vc:process (hsinc,reset)
begin
    if reset='0' then
        vcnt <= "000000000000";
    elsif hsinc'event and hsinc = '1' then
        if vcnt < 527 then
            vcnt <= vcnt + 1;
        else
            vcnt <= "000000000000";
        end if;
    end if;
end process;
```

Semnalul în ritmul căruia se realizează numărarea este chiar ieșirea `hsinc`.

e. Descrierea modului care asigură generarea semnalului de sincronizare pe orizontală `hsinc`

Modului testează starea semnalului `hcnt`, obținut de la numărătorul de sincronizare pe orizontală. Dacă acesta a ajuns la valoarea 1305 atunci semnalul de sincronizare pe orizontală este trecut în 0 logic. Acesta indică practic sfârșitul unei linii. Perioada acestuia este de fapt perioada de comutare a unei linii.

```

syh: process (clk,reset)
begin
    if reset = '0' then
        hsinc <= '1';
    elsif clk'event and clk = '1' then
        if hcnt>1305 and hcnt<1494 then
            hsinc <= '0';
        else
            hsinc <= '1';
        end if;
    end if;
end process;

```

Din motive legate de compatibilitatea cu toate tipurile de dispozitive de afișare, semnalul de sincronizare pe orizontală nu este ținut în 0 logic până la sfârșitul complet al numărării punctelor pe linie.

f. Descrierea modului care asigură semnalul vsinc de sincronizare a cadrelor

Perioada unui cadru este dată de semnalul Vsinc. Acesta este generat într-un modul care face testul semnalului vcnt și vede când numărarea acestuia se apropie de sfârșit. În acel moment dă semnalul vsinc în 0 logic.

```

syv: process (hsinc,reset)
begin
    if reset = '0' then
        vsinc <= '1';
    elsif hsinc'event and hsinc = '1' then
        if vcnt>494 and vcnt<496 then
            vsinc <= '0';
        else
            vsinc <= '1';
        end if;
    end if;
end process;

```

g. Descrierea circuitului de stingere

Circuitul de stingere urmărește starea semnalelor hcnt și vcnt generate de numărătorul de sincronizare pe orizontală și de numărătorul de sincronizare pe verticală. Atunci când valoarea acestora ajunge aproape de sfârșit înseamnă că apare o cursă inversă, pe orizontală sau pe verticală. Pe durata curselor inverse semnalul video pe ieșire trebuie „stins” adică să aibă valoarea care corespunde culorii negru.

--CIRCUITUL DE STINGERE

```

red <= "000" when (hcnt > 1248 or vcnt > 480 ) else red_int;
green <= "000" when (hcnt > 1248 or vcnt > 480 ) else green_int;
blue <= "000" when (hcnt > 1248 or vcnt > 480 ) else blue_int;

```

h. Descrierea memoriei care conține imaginea (memoria SRAM)

După inițializarea atributelor componentei BRAM, care a avut loc în partea declarativă a componentei, este momentul apelării componentei propriu zise.

```
----memoria BRAM
bram_img: RAMB16_S9 port map (
    DI => "11111111",
    DIP => "1",
    ADDR => adresa,
    EN  => '1',
    WE  => '0',
    SSR => '0',
    CLK => clk_div,
    DO  => data_out,
    DOP => datap_out
);
```

Aici sunt alocate semnalele pe porturile memoriei. Intrările de date (DI și DIP) nu sunt utilizate deoarece memoria nu se va scrie niciodată prin secvență de comandă (se încarcă inițial cu valorile imaginii). Memoria este permanent activată (EN este 1) dar are scrierea permanent dezactivată (WE este 0).

i. Descrierea numărătorului de adrese pe linie

Asigură numărarea pixelilor logici (cei care sunt diferiți în funcție de informația care o pot conține). Pe o linie avem un total de 32 de pixeli logici în acest exemplu.

```
numerator: process(clk_div, inceput_linie)
variable adr_int : std_logic_vector(4 downto 0);
begin
    if inceput_linie = '1' then
        adr_int := "00000";
    else
        if clk_div'event and clk_div = '0' then
            adr_int := adr_int + '1';
        end if;
    end if;
    adresa(4 downto 0) <= adr_int;
end process;
```

Fiecare pixel logic are în memoria BRAM o zonă (8 biți de date și bitul de paritate al 9-lea) în care se va afla informația de culoare a acestuia.

j. Descrierea numărătorului de adrese pe linii

Numărătorul de adrese pe linii are rol în contorizarea liniilor și generarea adreselor care vor fi accesate din memoria BRAM. Alături de numărătorul de adrese pe linie, acesta va genera la ieșire informația de adresă fizică. Ieșirea din numărătorul de puncte pe linie se concatenează cu cea a numărătorului de adrese pe linii.

```

numarator_linii : process(clkh_div, vcnt)
variable adr_int : std_logic_vector(5 downto 0);
begin
    if vcnt= "0000000000" then
        adr_int := "000000";
    else
        if clkh_div'event and clkh_div='1' then
            adr_int := adr_int + '1';
        end if;
    end if;
    adresa(10 downto 5) <= adr_int;
end process;

```

k. Descrierea divizorului de ceas pentru numărătorul de adrese pe linie

Pixelii fizici care se pot afișa pe o linie se împart la pixelii logici și anume informația distinctă, care se va afișa pe o linie. În cazul de față, raportul este de 1:39. Pentru a obține acest raport s-a constituit un divizor al semnalului de ceas care va ajunge apoi în numărătorul de adrese pe linie.

```

divizor_linie : process(clk,inceptu_linie)
variable cnt_int : integer;
begin
    if clk'event and clk='0' then
        if inceptu_linie = '1' then
            cnt_int := 0;
            clk_div <= '0';
        else
            cnt_int := cnt_int + 1;
            if cnt_int = 39 then
                cnt_int := 0;
                clk_div <= '1';
            else
                clk_div <= '0';
            end if;
        end if;
    end if;
end process;

```

l. Descrierea divizorului pentru numărătorul de adrese pe linii

Deși numărul de linii distincte posibile care s-au obținut este de 640 pe un cadru, în acest exemplu vor fi 20 de linii distincte care se pot afișa și care pot conține informație diferită. Din acest motiv, pentru acoperirea raportului de 1:32 s-a realizat un divizor al semnalului de ceas pentru numărătorul de adrese pe linii.

```

divizor_verticla : process(inceptu_linie,vcnt)
variable cnt_int : integer;
begin
    if vcnt= "0000000000" then
        cnt_int := 0;
    end if;
end process;

```



```

        clkh_div <= '0';
    else
        if inceput_linie'event and inceput_linie = '1' then
            cnt_int := cnt_int + 1;
            if cnt_int = 24 then
                cnt_int := 0;
                clkh_div <= '1';
            else
                clkh_div <= '0';
            end if;
        end if;
    end if;
end process;

```

m. Conectarea semnalelor pe ieșirile de culoare digitală

Cele trei semnale de ieșire de culoare digitală provin de la ieșirile memoriei BRAM. Ieșirea de date a memoriei (data_out) conține informații despre informația de culoare pentru roșu (bitul 2..0), pentru verde (bitul 2..0) și parțial pentru albastru (1..0). Bitul de paritate de pe ieșire (datap_out) conține informația de saturație pentru albastru (bitul cel mai semnificativ din informația digitală de albastru).

```

red_int <= data_out(2 downto 0);
green_int <= data_out(5 downto 3);
blue_int(1 downto 0) <= data_out(7 downto 6);
blue_int(2) <= datap_out(0);

```

n. Implementarea sistemului

După descrierea sistemului se efectuează sinteza și implementarea acestuia (vezi anexa). Pentru implementare trebuie alocate constrângerile legate de pinii fizici din FPGA care sunt conectați la portul VGA, la semnalul de ceas global și la butonul de reset.

Pentru aceasta se urmărește fișierul de constrângeri de mai jos:

```

NET "blue<0>" LOC = "c13" ;
NET "blue<1>" LOC = "a13" ;
NET "blue<2>" LOC = "e14" ;
NET "clk" LOC = "aa12" ;
NET "green<0>" LOC = "d10" ;
NET "green<1>" LOC = "c11" ;
NET "green<2>" LOC = "e11" ;
NET "hsinc" LOC = "a8" ;
NET "red<0>" LOC = "d9" ;
NET "red<1>" LOC = "d7" ;
NET "red<2>" LOC = "d6" ;
NET "reset" LOC = "y17" ;
NET "vsinc" LOC = "b14" ;

```

După implementare se vor obține diferite combinații de culori și se va schimba mesajul.

Temă

1. Să se identifice toate componentele prezentate și modul în care acestea au fost descrise în VHDL. Să se traseze o schemă detaliată a sistemului.
2. Să se afișeze textul având o nuanță de roșu saturat pe un fundal din verde cu chenar albastru.
3. Să se descrie un sistem capabil să afișeze la o rezoluție 800x600. Care este dimensiunea care trebuie alocată memoriei BRAM în acest caz.
4. Să se construiască sistemul care să permită afișarea animată a două cadre, ambele salvate în memoria BRAM

Criterii de evaluare:

Cerințe laborator	
1	7P
Temă	
1	1P
2	2P
3	2P
4	2P